SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Design And Analysis Of Algorithms **COURSE CODE:CSA0613**

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving. (BL5,BL6)

QUESTIONS: 1 **Total Marks:100**

Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I _P Hemanth(192472151)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **P Hemanth**

Q27. Sudoku Variation with Diagonal Constraints (Backtracking / NP-Complete Problem)

Solve a Sudoku puzzle where both diagonals must also contain unique numbers. Clearly define additional diagonal constraints. Analyze how constraints interact with standard Sudoku rules. Develop a backtracking-based solution. Apply constraint propagation and pruning. Justify correctness and discuss increased complexity.

Q27. Sudoku Variation with Diagonal Constraints

1. Problem Statement

Solve a Sudoku puzzle where both diagonals must also contain unique numbers. The program should define the additional diagonal constraints, analyze their interaction with standard Sudoku rules, and use a backtracking-based approach with constraint propagation and pruning.

---

2. Objective

The objectives of this program are:

- To solve a 9×9 Sudoku using backtracking.
- To add uniqueness constraints for both diagonals.
- To apply constraint checking and pruning.
- To analyze the correctness and complexity of the solution.

---

3. Introduction

Sudoku is a constraint satisfaction problem in which numbers from 1 to 9 must be placed in a 9×9 grid. Each number must occur only once in every row, column, and 3×3 sub-grid.

In this variation, two additional constraints are introduced. The numbers on the main diagonal and secondary diagonal must also be unique. This variation is commonly called **Diagonal Sudoku**.

The additional constraints make the puzzle more restrictive than standard Sudoku.

---

4. Additional Diagonal Constraints

Main Diagonal

The main diagonal contains:

(1,1), (2,2), (3,3), ..., (9,9)

All these cells must contain different numbers from 1 to 9.

Secondary Diagonal

The secondary diagonal contains:

(1,9), (2,8), (3,7), ..., (9,1)

All these cells must also contain different numbers from 1 to 9.

Standard Constraints

Every cell must satisfy:

1. No duplicate number in its row.
2. No duplicate number in its column.
3. No duplicate number in its 3×3 sub-grid.

Therefore, a number placed on a diagonal must satisfy both the normal Sudoku constraints and the corresponding diagonal constraint.

---

5. Algorithm Used

The program uses the **Backtracking Algorithm**.

Steps

1. Search for an empty cell.
2. Try numbers from 1 to 9.
3. Check whether the number is safe.
4. Check the row and column.
5. Check the corresponding 3×3 sub-grid.
6. If the cell is on the main diagonal, check the main diagonal.
7. If the cell is on the secondary diagonal, check the secondary diagonal.
8. Place the number if all constraints are satisfied.
9. Recursively solve the remaining cells.
10. If no solution is possible, undo the previous choice and try another number.

---

6. Constraint Propagation and Pruning

Constraint propagation is performed by checking all relevant constraints before inserting a number.

For example, if a candidate number already exists in the same row, it is immediately rejected.

Similarly, candidates are rejected if they already occur in:

- The same column
- The same 3×3 box
- The main diagonal
- The secondary diagonal

This process is called **pruning** because invalid branches of the search tree are eliminated early.

As a result, the algorithm does not need to explore every possible combination.

7. Execution

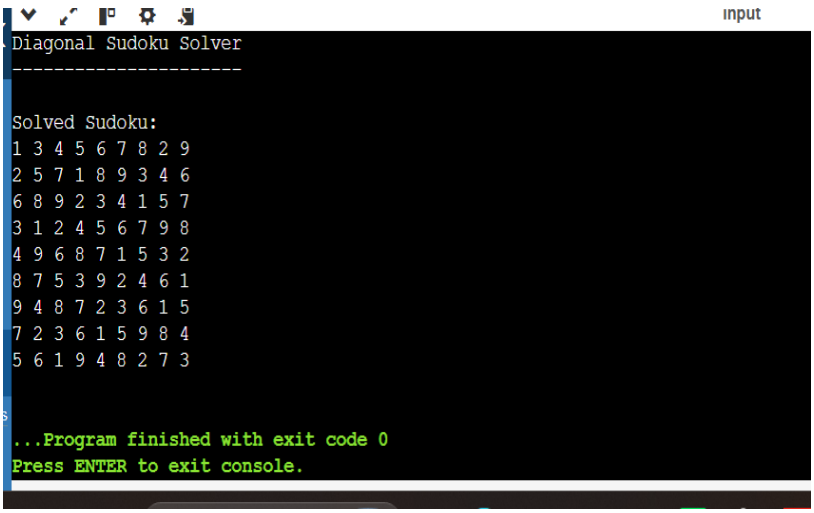
```
*main (3).py - C:\Users\neman\Downloads\main (3).py (3.13.15)*
File  Edit  Format  Run  Options  Window  Help
N = 9
def is_safe(grid, row, col, num):
    for i in range(N):
        if grid[row][i] == num or grid[i][col] == num:
            return False
    sr = row - row % 3
    sc = col - col % 3
    for i in range(3):
        for j in range(3):
            if grid[sr + i][sc + j] == num:
                return False
    if row == col:
        for i in range(N):
            if grid[i][i] == num:
                return False
    if row + col == N - 1:
        for i in range(N):
            if grid[i][N - 1 - i] == num:
                return False
    return True
def solve(grid):
    for row in range(N):
        for col in range(N):
            if grid[row][col] == 0:

                for num in range(1, 10):
                    if is_safe(grid, row, col, num):
                        grid[row][col] = num

                        if solve(grid):
                            return True

                        grid[row][col] = 0
                return False
    return True
def print_grid(grid):
    for row in grid:
        print(*row)
grid = [
    [0, 0, 0, 0, 0, 0, 0, 2, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 6],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
```

8. Output Analysis



9. Complexity Analysis

For a 9×9 Sudoku, there can be up to 81 empty cells and each cell can have up to 9 possible values.

In the worst case, the backtracking search can have exponential complexity:

Time Complexity:

O(9^81)

The diagonal checks add additional constraint checking compared with standard Sudoku.

However, the additional constraints can also improve pruning because more invalid candidates are eliminated before deeper recursion.

Space Complexity:

O(81)

This is mainly due to the Sudoku grid and recursive function calls.

10. Conclusion

The Diagonal Sudoku problem was successfully solved using a Python backtracking algorithm.

The solution extends standard Sudoku by requiring both diagonals to contain unique numbers. Constraint propagation and pruning are used to reject invalid candidates before continuing the search.

Although adding diagonal constraints increases the amount of checking, it also restricts the possible solutions and can reduce unnecessary search. The approach demonstrates how backtracking can effectively solve constraint satisfaction problems.